Transparent agentic science: Documentation, reliability, and scientific judgment

R. O'Shaughnessy
(Dated: October 23, 2026)

With the frontier-model capability considered here, adversarial artificial intelligence review already matches or exceeds typical human performance in finding and diagnosing errors in ordinary scientific workflows. Our development and operating experience includes multiple examples in code and results that had undergone extensive human scrutiny. We describe a living guide to reproducible scientific work developed through human–agent collaboration in gravitational-wave astrophysics. The guide addresses four concerns: traceable numerical documentation, reliable calculations, accurate attribution to the literature, and readable scientific arguments. Agents trace execution paths, construct independent calculations, and compare code, saved outputs, plots, and manuscript claims. Generated result macros preserve numerical provenance, adversarial review turns discrepancies into reproducible tests, and citation tools connect attributions to inspected sources. Human oversight remains essential throughout, particularly when physical intuition and plots reveal inconsistencies that automated checks miss. Writing presents the largest continuing difficulty: fluent and abundant prose can obscure priorities, context, and the central scientific contribution. Strong models, sustained revision, and explicit human direction are needed to improve the argument itself. The examples concern gravitational-wave population papers and the RIFT development and review sequence, connected to its third-generation methods paper. The accompanying public repository is assembled entirely with AI from prior work with a human, and all its commits are performed by AI agents.

I. INTRODUCTION

The use and capabilities of artificial intelligence (AI) in science are expanding rapidly. Studies of the research literature document accelerating adoption across scientific fields [1], while demonstrations of language-model systems connected to software and laboratory tools show how their role can extend from text generation to executing research tasks [2]. In computational physics, an agent can translate a scientific question into code, run an analysis, inspect its output, and propose a revision within one working session. For a developer or operator, the immediate benefit is the ability to investigate a failure across the whole analysis: inspect the implementation, reproduce the behavior, construct a counterexample, and check every dependent result. In our experience, agents repeatedly find bugs and inconsistencies that survived extensive human development and review. This changes the practical standard of care: an established codebase or familiar result should be actively challenged with these additional capabilities.

Journal and preprint policies identify several concerns that bear directly on this challenge. The Physical Review policy permits substantive AI assistance with disclosure, while retaining author responsibility for accuracy and verification [3]. The guidance for the Astrophysical Journal (ApJ) and other American Astronomical Society (AAS) journals emphasizes supervision, disclosure, original work, and checks on data, figures, and references; it specifically warns that generated reference lists can be incomplete or fictitious [4]. arXiv likewise requires significant use of generative language tools to be reported and holds each author responsible for the content [5]. These concerns address concrete failure modes of poor or unchecked use: invented evidence, unreliable calculations, misleading attribution, and plausible prose that has not received scientific scrutiny. Disclosure makes the process visible, but a disclosed error remains an error.

In this paper, we present a living guide to conducting science reliably with agents. The guide translates these concerns into practices for documenting results, validating calculations, consulting the literature, and constructing a scientific argument. It draws on human–agent work on gravitational-wave population inference, illustrated by the main synthetic-universe and hierarchical-modeling papers [6, 7], the RIFT pull-request review sequence, and the third-generation RIFT study [11], together with a collection of reusable research skills. Our aim is to use the agents' substantial advantage in systematic bug finding to improve the reliability of scientific work, while retaining human responsibility for its questions, interpretation, and presentation. The guide will change as tools improve and experience exposes new failure modes. The assessment draws on repeated developer and operator experience, including multiple investigations in which agents located and diagnosed errors that previous human scrutiny had missed.

Unless otherwise noted, throughout this guide we use the latest OpenAI model available at the time of the work, at Sol scale, or a comparably capable Claude model at Opus scale. This convention refers to the model's corresponding capability at that time, rather than a fixed historical version. We recommend comparable capability for scientific work. Under these conditions, adversarial agent review already performs at or above typical human review in finding and diagnosing errors in ordinary scientific workflows; we have multiple examples from the development and operation of the analyses discussed here.

A useful division of responsibility emerges from this experience. Agents are particularly strong at following a

discrepancy through code, enumerating neglected cases, and checking whether a claimed fix reaches every caller and output. The human scientist contributes context, physical intuition, and decisions about which discrepancies matter for the research question. Either can originate the objection, and either can be wrong; a reproducer or independent calculation resolves the issue. The largest remaining imbalance is in writing: an agent that is excellent at exposing a defect may still misjudge the central argument of the paper.

This paper is organized as follows. Section II discusses numerical documentation and provenance. Section III considers reliable computation, adversarial review, and human oversight. Section IV describes literature retrieval and citation checks. Section V addresses scientific writing, the component that currently requires the most sustained judgment. Section VI summarizes the approach. Appendix A provides a small executable illustration of numerical consistency, and Appendix B identifies the accompanying resources.

II. DOCUMENTING NUMERICAL EVIDENCE

A quantitative statement must refer to an identifiable calculation. A language model's ability to produce a plausible number is irrelevant to whether that number was measured or computed. The analysis must produce a saved result, with sufficient information to recover its inputs, method, uncertainty, and domain of validity. We organize this documentation around the connection from data to computation to scientific claim. This approach is modeled on the provenance and review practices used in LVK internal review, adapted here to research conducted with agents. The public LVK data-release documentation provides a related example of connecting reported quantities to identifiable data products [14].

The essential mechanism is a generated numerical macro. The analysis writes a result record containing the raw value, units, uncertainty definition, input and configuration identities, and producing code. A deterministic tool converts this record into the quantities displayed in the manuscript. The abstract, text, table, and caption then refer to the same named result. A change to that result propagates to every use when the document is rebuilt. The same record can supply table cells and numerical annotations in plots, eliminating separate transcriptions that otherwise become inconsistent during revision.

Provenance must describe the calculation that actually ran. A source comment, directory name, or remembered command may describe an intended configuration rather than the effective configuration. Relevant settings therefore belong in the output itself, alongside the input identities, code revision, environment, seed where applicable, and termination status. Cached products require the same care: a calibration or selection change can invalidate a cache without changing its filename. Their identity must include the dependencies that determine their contents. Optimization results also require the initialization, restart schedule, checkpoints, and rule used to select the reported solution.

Documentation has a scientific purpose beyond bookkeeping. A historical comparison may intentionally retain its original sample, whereas a current inference must use the current analysis inputs. Both can be valid, provided that their domains are stated. A stale result, an explicitly frozen historical result, and a curated input constant are therefore distinct. Treating all older numbers as interchangeable invites errors: automatically updating a historical comparison can be as misleading as retaining a superseded measurement.

Tooling checks the consistency of this chain. A clean build regenerates numerical products and compares them with the recorded artifacts, identifies undefined result names, and flags computed quantities that bypass the result record. It must also fail when its required tools are absent. These checks prevent specified forms of transcription drift; they do not establish that the estimator or scientific interpretation is correct. Selecting the wrong result key remains a scientific error even if that key is used consistently throughout the paper. Appendix A illustrates both the utility and the limited guarantee of the mechanism.

Every figure and repository needs a reproduction route. A figure's caption should link to its recipe and provenance: input identities, cuts and transformations, plotting code, effective configuration, environment, plotted numerical data, and final artifact identities. Distinguish replotting saved data from rerunning the analysis. The repository entry point should identify how to obtain the inputs, run representative calculations, and verify outputs, including access restrictions, resource requirements, tolerances, and archived expensive runs. The `result-provenance` skill supports these records and their generated products.

The manuscript and its supporting notes serve different readers. Scientific notes retain derivations, rejected approaches, uncertainty studies, and the evidence behind changes of interpretation. The paper selects the material needed to understand and assess its claims. Keeping that distinction allows documentation to remain detailed without forcing the reader to reconstruct the argument from a chronological account of runs. The research record can be extensive while the manuscript remains focused.

III. ESTABLISHING RELIABLE RESULTS

A. Tools, comparisons, and uncertainty

At the Sol/Opus capability assumed here, bug finding and diagnosis are demonstrated strengths. Adversarial agent review is already at or above typical human performance for these tasks in ordinary scientific workflows.

In our experience as developers and operators, agents are often substantially more effective at finding implementation defects and inconsistencies between calculations, outputs, and scientific claims. This includes mature code and results that have already been examined extensively by humans. Agents can sustain a detailed audit across files, interfaces, configurations, and artifacts, then execute the small experiments needed to resolve each suspicion. Extensive previous review provides useful context for this work; it should not exempt a component from another audit.

The advantage is concrete. If two solvers disagree, an agent can reduce the problem to a common input, match units and normalization, compare intermediate quantities, and identify the first operation where they diverge. If an argument is added to an interface, it can enumerate the callers and find the launcher that still supplies the old default. If a result changes after a calibration update, it can inspect cache keys and demonstrate that an old artifact was reused. If a manuscript and a plot disagree, it can trace both back to their saved outputs and identify the run that supplied each. These tasks demand exhaustive attention to details that are easy for a human reviewer to miss.

The main-paper development records contain examples of precisely these failures. An agent caught a sample cache that assigned data to the wrong event by comparing the physical properties of the cached samples with the catalog labels, then added an invariant check before writing the cache. Another investigation compared a saved physical configuration with the claimed configuration and exposed a launcher default that had propagated into figures and numerical summaries. These checks examine the relationship between an output and what it purports to represent. A successful fit, a plausible plot, and a provenance comment can all coexist with the defect.

Evidence from established software infrastructure supports taking this capability seriously beyond these projects. Google reported that AI-generated fuzz targets exposed previously unknown vulnerabilities in OpenSSL and other projects already subjected to extensive fuzzing. In cJSON, an AI-generated target found a defect in a function that already had a human-written fuzz harness [13]. The important point is additional discriminating coverage: mature software and substantial previous testing still contain cases that a new agent-generated check can expose.

A useful bug report therefore contains the input or execution path, the observed discrepancy, the violated expectation, and a reproducer. The agent tests the proposed repair against that reproducer, checks related call sites, and preserves a regression test. For a result inconsistency, it also enumerates the figures, tables, and claims that depend on the affected output. This is the deliverable that makes automated criticism valuable: a scientist can inspect and rerun the evidence, rather than assess a list of plausible objections.

Validation is most informative when its alternatives fail for different reasons. Analytic answers and planted-truth examples check known outcomes. Limiting cases test physical conventions and normalization. Refinement studies expose approximation error, and independent seeds characterize stochastic variation. A second implementation can identify mistakes in the first, provided that their shared dependencies and assumptions are understood. The acceptance criterion should be chosen from the scientific question and its error budget, rather than adjusted after seeing the comparison.

For inference, reliable execution must be separated from reliable interpretation. A sampler can mix well while the parameter of interest remains unidentified or pressed against a prior boundary. A global likelihood improvement can coexist with failure in the subpopulation that motivates the claim. The main hierarchical-modeling paper illustrates both issues: it distinguishes a boundary-limited result from a measurement and uses a detected-population comparison to restrict the interpretation of a fitted model [7]. The relevant test follows the claim into the space where it has an observable consequence.

Finite numerical support deserves particular attention. Scatter across seeds does not by itself establish convergence with sample size, and more optimizer iterations cannot repair a simulation library that does not populate the relevant region. The main synthetic-universe paper accordingly treats an additional population component as a hypothesis when its available numerical support does not permit a reliable evidence comparison [6]. Recognizing such a limitation can be more useful than constructing another estimator around the same insufficient inputs.

B. Continuous integration and production validation

Adversarial findings should become executable checks. CI should exercise the operator's entry point, configuration propagation, numerical answers and boundaries, and generated artifacts. Restore a diagnosed defect in an isolated copy to show that its regression test fails. Record what actually ran: collected, passed, failed, and skipped checks, the code and environment identities, and missing hardware or dependencies. A green badge cannot establish an untested scientific claim.

Validate new code against scientific examples and established references. Unit tests are followed by comparisons that an independent scientist can interpret:

- **Reproduce the paper examples.** Run the documented demonstrations and representative analysis cases; regenerate their observables, figures, and tables. Check the numerical claims they support, including their uncertainty and stated domain.

- **Compare with previous codes.** Run the new and earlier released implementations on matched

inputs, physical conventions, settings, and accuracy targets. Explain disagreements; preserve intentional corrections as versioned changes with regression evidence.

- **Use external benchmarks.** Reproduce analytic results and benchmark calculations from the literature with their published assumptions and error budgets. Identify the reference, implementation, and any shared dependencies so agreement is meaningful.

- **Exercise production environments.** Validate the installed executable and normal launch/configuration route on the intended CPU/GPU backend, dependency stack, scheduler, and representative data. Include a bounded end-to-end case and comparison with a baseline; record missing coverage and obtain the full production rerun when needed.

A fast CI fixture should preserve a small version of each practical regression where possible. Expensive paper reproductions and production campaigns retain archived commands, environment identities, raw outputs, tolerances, and a receipt linked from the change. Passing a helper test on a laptop does not establish that the deployed workflow reproduces the paper or the benchmark. An older code is a comparator, not an infallible oracle: resolve a disagreement through independent evidence rather than forcing agreement with a known defect.

The `simulation-validation` skill supports these comparison routes and validation receipts. CI preserves recurring checks; separately archived campaigns supply the evidence for expensive production cases.

C. Concrete examples from RIFT development

The RIFT pull-request sequence provides a particularly useful scientific-software example because it preserves the diagnosis, proposed repair, review findings, and tests together. The cases below are merged repairs in the research development repository. They concern production likelihood behavior, the correctness of a new numerical method, and agreement between execution backends. The linked records describe the reproduced failures and checks; the discussion here does not claim to have repeated those computational campaigns.

In the vectorized time-marginalization path, a single maximum over a batch was used to stabilize exponentiation. In exact arithmetic, the common offset cancels when the integral is converted back to a log likelihood. In floating-point arithmetic, a loud sample can set an offset that drives ordinary, lower-likelihood samples in the same batch to underflow. The investigation reproduced the defect on both array backends and corrected the offset separately for each sample [8]. Regression tests included a square array, where an incorrect broadcasting repair would silently compute the wrong result. This is a concrete error in an established likelihood calculation whose algebra appears correct on inspection.

Review of a log-space Hermite quadrature method exposed several different failures while existing tests passed [9]. The implementation rejected valid rows containing finite values and negative infinity, its endpoint treatment contradicted the stated polynomial-reproduction property, and a test fixture could not distinguish the claimed behavior from an incorrect alternative. Further checks found that the command-line parser could not select the new method and that deleting its dispatch branch left the earlier tests green. The repairs therefore covered the numerical method, its tests, and the route by which an operator actually requested it. Deliberately restoring the defects made the new checks fail.

A subsequent CPU/GPU comparison found that the same integration-rule name selected different numerical rules for the same sampled likelihood [10]. The repair made rule selection explicit and consistent across backends, then tested the disagreement by reinstating the old behavior. Together, these cases demonstrate why agent review is valuable after a code review or a passing test suite: it can challenge the floating-point realization of an identity, the discriminating power of a test, the reachability of a feature, and consistency between operating paths.

The scientific context is developed in the third-generation RIFT methods paper [11]. Loud signals, narrow likelihood peaks, and time- and frequency-dependent detector responses make numerical resolution and implementation consistency scientifically consequential. The posted main paper supplies that context; the PR sequence supplies concrete development evidence, including later repairs that should not automatically be attributed to the posted version. Forthcoming studies and subsequent revisions can extend the scientific tests and connect further diagnoses to their consequences. Completed review records and future investigations retain their respective status.

D. The matter backend and operational lessons

The nuclear-matter backend shows why independent solvers and production tests matter. In NuclearMatter-Backend (`nmbackend`), an independent radius-domain stellar-structure solver exposed under-resolution in an enthalpy-domain default and motivated an adaptive implementation. Subsequent comparisons with RePrimAnd and other reference routes distinguish solver error from interpolation and surface conventions [12]. Value agreement alone does not validate derivatives: the development record calls for separate automatic-differentiation and finite-difference checks at the derivative order used by the analysis.

Read-only project lessons from CIT, cardassia, and sporc add an operational test: **verify the code ac-**

tually imported and the workflow actually completed. Relevant failures include repeated JAX compilation under a production resource limit, an environment importing a different checkout, a shared branch moving between validation and delivery, and a completed long job whose follow-on step never ran. These records motivate import-path and revision checks, repeated-call tests, scheduler-aware execution receipts, and revalidation on the delivered revision. They are development lessons from restricted sources, not newly reproduced campaigns or claims that all remaining validation gaps are closed.

E. Adversarial review and human oversight

Adversarial review deliberately directs the agents' bug-finding ability at an established result, including one the implementer and human reviewers believe is correct. We distinguish a science review of the claim, assumptions, and domain from a code and numerics review of the implementation and its checks. Both examine a fixed candidate and a stated scientific specification. They seek counterexamples, convention errors, confounded comparisons, and unjustified extrapolations, with enough information to reproduce the objection. A reviewer should know the question and assumptions without being primed by the implementer's preferred explanation of why the answer must be correct.

The independence of a check depends on its evidence as well as its reviewer. Separate agent contexts can reduce agreement driven by a shared conversation, but they do not remove a common library error, input defect, or modeling assumption. The record should state which route was independent and for what purpose. A case used to repair or select the candidate becomes development evidence; a subsequent confirmation needs fresh cases where feasible. Deliberately incorrect inputs or isolated mutations can show that a check is capable of rejecting a relevant failure.

Human oversight and agent auditing are complementary. Physical intuition can identify an anomaly before anyone knows which test to write; the agent can then pursue its cause across the implementation and results. A plot can make an implausible boundary behavior, displaced population, missing tail, or incorrect normalization immediately apparent. Such a judgment should initiate a test: identify the expected behavior, compare it with the plotted quantity, and trace any discrepancy through inputs, transformations, selection effects, and visualization code. The scientist's intuition is a source of hypotheses, not a desired answer that the agent must reproduce. A useful agent can also demonstrate when that intuition was wrong.

Plots therefore need direct scientific inspection, not merely confirmation that an image file exists. Axes, units, cuts, normalization, uncertainty bands, and the distinction between a likelihood and a probability density can all change the apparent conclusion. A summary statistic may conceal a shape discrepancy that is obvious in an overlay or residual plot. Visualization code must read the recorded data and preserve its meaning; an image generated to look like a scientific plot supplies no evidence. The AAS guidance makes the same practical distinction between supervised visualization of supplied data and invented scientific graphics [4].

Review also governs revision. Findings remain recorded with their dispositions, including unresolved disagreement and objections rejected with evidence. When a claim fails, its supporting note is marked accordingly, the defeating test becomes a standing diagnostic, and every dependent statement is checked. The abstract, captions, conclusions, and related main paper can otherwise preserve an argument that the calculation no longer supports. This sweep is repeated after the final human or collaborator edit. Human involvement does not make a revision immune to staleness.

IV. FINDING AND CITING RELEVANT LITERATURE

Scientific attribution requires both a real source and a correct account of what it establishes. A model can invent a bibliographic entry, merge details from different papers, or attach a genuine citation to a claim that the cited work does not support. It can also supply a superficially credible but incomplete account of a field. The solution begins with the same tools and habits used by careful human researchers: search the literature, retrieve the source, inspect the relevant passage, and preserve its bibliographic identity.

Agents should use citation and discovery services such as ADS, arXiv, DOI resolution, and journal records. A reference manager can import verified metadata, while simple checks identify duplicate entries, undefined citation keys, missing identifiers, or mismatches in author, title, and year. These checks establish identity. Scientific support requires reading the cited equation, result, or discussion in context, including assumptions and limitations. A generated summary is a starting point for inspection rather than evidence that the source was read.

Relevance and coverage require additional judgment. Searches should follow the scientific ideas, methods, and competing interpretations rather than only the vocabulary suggested by the current draft. Citation chains and domain expertise help recover work that a generated list misses. The ApJ/AAS guidance explicitly recommends conventional literature tools, including ADS, alongside checks of whether suggested references exist and are relevant [4]. These are ordinary scholarly responsibilities; using an agent makes them easier to automate in part without making them optional.

The same standard applies to this guide. Journal requirements are cited to the current official policies, and the scientific examples refer to the main papers in their respective repositories. Repository links identify the sup-

porting research context but currently require access; the preprints provide public scientific sources. We do not infer a claim's validity from the existence of either a repository or a preprint. The relevant evidence must still support the sentence for which it is cited.

V. WRITING A SCIENTIFIC ARGUMENT

Writing is currently the part of this process that requires the most judgment. Agents can generate large amounts of grammatical prose quickly, yet that prose may give the wrong material the greatest weight. A draft can dwell on implementation minutiae, repeat caveats, miss the context that motivates the work, or describe a sequence of calculations without identifying the scientific contribution. These are failures of argument and priority. Correcting the wording alone leaves the underlying problem intact.

The human author must supply an explicit scientific brief: the question, audience, principal contribution, important comparisons, limits of the evidence, and intended thrust. This brief determines what belongs in the paper and what belongs in notes or supporting documentation. The agent can then help expose gaps, propose a structure, challenge a transition, or draft a bounded passage. The direction of work is from the scientific argument to the prose. A collection of polished paragraphs does not become a paper merely by placing headings between them.

Our practical recommendation is to use the frontier capability specified in the Introduction, or demonstrated comparable capability, and to reserve substantial time for revision. The assessment of adversarial review above should not be transferred automatically to smaller or less capable models. Model quality matters for sustained reasoning about context and the relationship between sections; it does not eliminate the need to supply that context. The author may need to explain repeatedly why a particular result matters, why another belongs in an appendix, or why the apparent novelty is already established in the literature. These recommendations reflect experience in this collaboration, rather than a measured ranking of models or a guarantee about future systems.

Revision should first ask whether the argument is worth reading. The introduction must connect the broader scientific problem to the contribution. Methods must explain the assumptions and formalism needed to assess the result. Results must distinguish what was learned from what was merely run. Discussion must address implications and limitations at the appropriate scale. The abstract and conclusions should state the same central finding with a strength supported by the evidence. This is where agents often need the most guidance: local correctness can coexist with a mistaken view of the paper's purpose.

Only after the argument is coherent does line editing become efficient. Useful passes remove repeated material, replace vague claims with concrete comparisons, shorten descriptions of machinery, and improve the connection between paragraphs. They preserve numbers, units, uncertainty definitions, notation, and attribution. A proposed stylistic change that strengthens a scientific claim is a substantive revision and must be reviewed as such. Repeated generation without a clear diagnosis tends to produce more text instead of a better explanation.

A. Guardrails and guidance

Bound the writing task before generating prose. Open-ended writing instructions produce stochastic output: even a request to tighten an argument can change its emphasis or intent. In our experience, this instability increases with the size of the block assigned to the model. This revision supplies a concrete example. A request to tighten this guide produced a comprehensive rewrite, including a new title and abstract, that the human author judged to have changed the intended framing. Restoring the earlier draft and reviewing bounded additions was necessary to recover that intent. This is a documented editing failure, not a measured scaling law for model behavior.

We recommend the following guardrails:

- **Human-guided structure.** Establish the thesis, audience, outline, and role of each section before drafting. Preserve an accepted title and abstract unless their revision is explicitly requested.
- **Bounded initial drafts.** Assign one subject or argument with stated evidence, exclusions, and intended scope. Ask for an expansion of the outline separately before authorizing a larger draft.
- **Narrow revisions.** Edit section by section, naming the defect to fix and the claims and framing to preserve. An instruction to shorten or clarify is not permission to replace the paper's central argument.
- **Redline validation.** Preserve the accepted version and inspect the changes before accepting each edit. Check additions and deletions for shifts in intent, emphasis, claim strength, citation scope, notation, and evidence. A successful compile checks typesetting; it cannot validate those semantic choices.

The `human-directed-science-writing` skill should enforce this bounded workflow. Broader restructuring is a separate proposal for human review. Once a section is accepted, use it as the baseline for the next narrow edit rather than repeatedly regenerating the whole manuscript.

The human author remains responsible for the final document, including passages the agent drafted. The goal is a paper whose claims, priorities, and reasoning the author can explain under questioning. In that sense,

writing is also a test of scientific understanding: difficulty explaining the main point may reveal an unsettled interpretation or an unnecessary calculation. Effective assistance improves the argument while preserving this connection between the scientist and the work.

VI. CONCLUSIONS

We have described a living approach to reliable scientific work with agents, organized around numerical documentation, computational reliability, literature attribution, and scientific writing. Generated result macros and provenance records make numerical claims traceable and prevent defined forms of transcription drift. At the model capability assumed here, adversarial review already matches or exceeds typical human performance at finding and diagnosing errors, including defects and inconsistencies that extensive human vetting has missed. Tool execution, independent comparisons, and adversarial review turn that ability into reproducible evidence and durable repairs. Citation tools and direct reading connect claims to genuine and appropriate previous work. These practices build on familiar standards of scientific work and review.

Human oversight determines the scientific priorities and interpretation, while agents provide unusually effective, persistent scrutiny of code and results. Physical intuition and inspection of plots can initiate an investigation; an agent can also originate the objection, demonstrate the defect, and trace its scientific consequences. Writing remains the least mechanical component: it requires context, priorities, capable models, sustained revision, and an author who owns the scientific argument. The guide and its supporting repository will evolve as those capabilities and practices improve.

ACKNOWLEDGMENTS

This working manuscript and its public repository were assembled with OpenAI Codex under human direction, using previous human–agent research work and the explicit argument supplied for this revision. The agent retrieved policy and literature sources, reorganized and drafted text, and implemented and checked the accompanying synthetic tooling. All repository commits are performed by AI agents. This disclosure records the work performed; final human review and approval remain necessary before submission. Detailed model-version information and a final verification record will accompany a submitted version.

DATA AND CODE AVAILABILITY

The living guide, source, portable skills, and synthetic example are available in the public companion repository. The two scientific examples are the main papers in popsynth_hyperpipe_paper and gwk-hierarchical-initial; both repositories currently require permission to access. Their main preprints are publicly available [6, 7]. The additional RIFT examples link to the currently access-restricted RIFT development PR sequence and RIFT methods-paper repository; the third-generation main paper is public [11]. The public NuclearMatter-Backend supplies the additional code example and dated validation records. The tooling source agent-skills also currently has restricted access. Public textual examples are not represented as new end-to-end replications of the scientific analyses.

TABLE I. Synthetic consistency example. The result record generates the table and numerical prose together; it records input and producer identities. OpenAI Codex implemented the generator and its tests under human direction.

Quantity	Value (dimensionless)
Midpoint estimate	0.33332825
Antiderivative reference	0.33333333
Absolute discrepancy	0.00000509
Acceptance threshold	0.00001000

Appendix A: An executable consistency example

A small synthetic example isolates the numerical documentation mechanism from the astrophysical applications. It evaluates

$$I = \int_0^1 x^2 \, \mathrm{d}x \tag{A1}$$

by composite midpoint quadrature and an antiderivative. Exact rational accumulation removes stochastic variation from the quadrature. The shared mathematical specification and runtime limit the independence of the comparison.

The estimate is 0.33332825, with absolute discrepancy 0.00000509 below the prescribed tolerance 0.00001000 (Table I). These quantities are inserted from the same generated record. Within a declared restricted TeX grammar, tests deliberately change inputs and producer identity, corrupt generated artifacts, introduce an undefined result or untracked source, and insert a typed numerical result. A wrong calculation must fail the numerical acceptance test, while refinement must improve the approximation in the expected way. This demonstrates specific checks, not a general proof of scientific correctness or a benchmark of agent performance.

Appendix B: Living resources

The public repository separates the scientific guide from its operational material. The latter includes

the earlier agent-facing document, reusable skills for provenance, simulation validation, adversarial review, and scientific writing, and the synthetic example's regeneration and checking tools. The skills are `result-provenance`, `simulation-validation`, `adversarial-science-review`, and `human-directed-science-writing`. These resources support the argument in the paper without requiring the reader to follow an agent's execution history. New lessons can be added with their evidence and limitations, while scientific claims remain tied to a particular reviewed version.

[1] Q. Hao, F. Xu, Y. Li, and J. Evans, Artificial intelligence tools expand scientists' impact but contract science's focus, Nature **649**, 1237–1243 (2026).

[2] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, Autonomous chemical research with large language models, Nature **624**, 570–578 (2023).

[3] American Physical Society, Appropriate Use of AI Tools, Physical Review editorial policy (accessed October 23, 2026).

[4] American Astronomical Society, Author Guidelines for Use of AI and LLMs in Manuscript Preparation (initially posted September 9, 2026; accessed October 23, 2026).

[5] arXiv, Policy for authors' use of generative AI language tools, Content Moderation (accessed October 23, 2026).

[6] R. O'Shaughnessy, R. Mechum, M. Qazalbash, Z. Rosenberg, and M. Zeeshan, Physics-based phenomenological modeling of binary black hole hierarchical formation 1: Synthetic universes from globular cluster simulations for GWTC, arXiv:2609.05996 (2026).

[7] R. O'Shaughnessy, M. Zeeshan, and M. Qazalbash, Physics-based phenomenological modeling of binary black hole hierarchical formation 2: Autodifferentiable functional inference of hierarchical compact-binary populations, arXiv:2609.06728 (2026).

[8] RIFT development record, PR 234: Offset the vectorized time marginalization per extrinsic sample, merged September 3, 2026 (access restricted).

[9] RIFT development record, PR 266: Log-space cubic-Hermite time quadrature, merged October 5, 2026 (access restricted).

[10] RIFT development record, PR 459: Time marginalization: one rule per name on CPU and GPU, merged October 18, 2026 (access restricted).

[11] R. O'Shaughnessy, A. Jan, J. Lange, and R. Mechum, Scaling RIFT 1: Extending RIFT to third-generation gravitational-wave analyses, arXiv:2609.37393 (2026).

[12] NuclearMatter-Backend, dated validation-source inventory and linked public code documentation. Additional operational lessons were inspected in restricted project records.

[13] O. Chang, D. Liu, and J. Metzman, Leveling Up Fuzzing: Finding more vulnerabilities with AI, Google Open Source Security Team (November 20, 2024).

[14] Gravitational Wave Open Science Center, GWTC-3 Data Release Documentation.